EX05_LapKeHoach_Va_DoThiKeHoach

Advanced Artificial Intelligence



GV HƯỚNG DẪN : THẦY ĐỖ NHƯ TÀI

HỌ VÀ TÊN SV: BÙI MINH TRUNG

MSSV-LỚP: 3124720074 – DNT1241

Bài 1. Khái niệm cơ bản trong GraphPlan

a) Interference (xung đột do can thiệp):

Hai hành động ở CÙNG một mức hành động A_i bị mutex do Interference khi delete list của hành động này chứa một precondition (hoặc một effect) của hành động kia. Nói cách khác, nếu thực hiện đồng thời, một hành động sẽ "phá" mất điều kiện mà hành động còn lại cần để hợp lệ. Ý nghĩa: đây là cơ sở để GraphPlan xác định những hành động KHÔNG thể xếp song song trong cùng một bước thời gian. **b) Inconsistent Effects (hiệu ứng không nhất quán):**

Hai hành động bị mutex do Inconsistent Effects khi effect (add hoặc delete) của hành động này là phủ định trực tiếp của effect hành động kia trên CÙNG một literal — ví dụ một hành động thêm P trong khi hành động kia xóa P (hay thêm ¬P). Ý nghĩa: nếu cho cả hai chạy song song, trạng thái kết quả sẽ mâu thuẫn (P vừa đúng vừa sai), nên GraphPlan đánh dấu cặp này không thể cùng xuất hiện ở một mức. **c) Competing Needs (tiền điều kiện cạnh tranh):**

Hai hành động bị mutex do Competing Needs khi tồn tại một precondition của hành động này mutex với một precondition của hành động kia tại mức trạng thái S_i liền trước. Ý nghĩa: dù bản thân hai hành động không trực tiếp phá effect của nhau, chúng vẫn không thể cùng tồn tại vì các điều kiện tiên quyết của chúng vốn loại trừ lẫn nhau.

d) Các cách tìm một kế hoạch trong mô hình lập kế hoạch cổ điển:

- Tìm kiếm tiến (forward/progression search) — duyệt tiến từ trạng thái đầu qua không gian trạng thái.
- Tìm kiếm lùi (backward/regression search) — duyệt lùi từ tập mục tiêu.
- Lập kế hoạch dựa trên đồ thị kế hoạch (GraphPlan) — xây planning graph rồi trích xuất kế hoạch.
- Chuyển bài toán planning thành công thức logic mệnh đề và dùng SAT solver (SATPlan).
- Tìm kiếm có heuristic (ví dụ heuristic trích từ planning graph như FF/relaxed-plan heuristic).
- Lập kế hoạch từng phần có thứ tự bộ phận (Partial-Order Planning).

e) Giả định được nới lỏng khi dùng heuristic từ GraphPlan:

Giả định về Delete List được nới lỏng — heuristic thường được tính trên một "bài toán nới lỏng" (relaxed problem) trong đó mọi hành động được giả sử KHÔNG xóa gì cả (bỏ qua delete effects). Nhờ bỏ delete, planning graph trở nên đơn điệu (một literal đã xuất hiện sẽ không bao giờ biến mất), nên có thể xây dựng rất nhanh và số mức để các mục tiêu xuất hiện (level mục tiêu đạt được) trở thành một ước lượng admissible cho chi phí còn lại. Điều này hữu ích hơn tìm kiếm ngây thơ (không có heuristic, phải mở rộng mù toàn bộ không gian trạng thái) vì nó cho thuật toán tìm kiếm một "la bàn" ước lượng khoảng cách tới đích, giúp định hướng và cắt tỉa mạnh không gian tìm kiếm.

Bài 2. Lập kế hoạch ký hiệu — Bài toán cần cẩu

a) Biến/đối tượng, predicate, trạng thái đầu và mục tiêu

- Đối tượng: Crane (cần cẩu, hằng số duy nhất), Cargo (kiện hàng, hằng số duy nhất), Truck (xe tải, hằng số duy nhất); hai vị trí Left, Right.
- Predicate: CraneAt(loc) — cần cẩu đang ở vị trí loc; CargoAt(loc) — kiện hàng đang ở vị trí loc (khi chưa lên xe); TruckAt(loc) — xe tải ở vị trí loc; Empty(Crane) — cần cẩu đang rảnh tay; Holding(Crane,Cargo) — cần cẩu đang cầm kiện hàng; On(Cargo,Truck) — kiện hàng đã ở trên xe tải.
- Trạng thái ban đầu S0: CraneAt(Left) ∧ CargoAt(Left) ∧ TruckAt(Right) ∧ Empty(Crane).
- Trạng thái mục tiêu: On(Cargo,Truck) ∧ CraneAt(Left).

b) Các toán tử hành động

Toán tử	Preconditions	Add List	Delete List
Move(from,to)	CraneAt(from)	CraneAt(to)	CraneAt(from)
PickUp(loc)	CraneAt(loc), CargoAt(loc), Empty(Crane)	Holding(Crane,Cargo)	CargoAt(loc), Empty(Crane)
LoadOnTruck(loc)	Holding(Crane,Cargo), CraneAt(loc), TruckAt(loc)	On(Cargo,Truck), Empty(Crane)	Holding(Crane,Cargo)

(Xe tải đứng yên tại Right trong suốt bài toán vì đề không yêu cầu di chuyển xe tải; chỉ cần cần cẩu di chuyển. LoadOnTruck chỉ hợp lệ khi loc = Right, vì đó là vị trí của xe tải.)

c) Planning graph cho 3 bước đầu (S0 → A0 → S1 → A1 → S2)

Ký hiệu: L=CraneAt(Left), R=CraneAt(Right), CL=CargoAt(Left), TR=TruckAt(Right), E=Empty(Crane), H=Holding(Crane,Cargo).

Mức	Nội dung
S0	L, CL, TR, E
A0	Move(Left,Right); PickUp(Left); Noop(L), Noop(CL), Noop(TR), Noop(E)
S1	L, R, CL, TR, E, H (R từ Move; H từ PickUp; L,CL,E vẫn còn nhờ Noop)
A1	Move(Left,Right); Move(Right,Left); PickUp(Left); LoadOnTruck(Right); Noop cho mọi literal ở S1
S2	L, R, CL, TR, E, H, On(Cargo,Truck) (On xuất hiện từ LoadOnTruck)

Ở S2, mục tiêu On(Cargo,Truck) đã xuất hiện; literal CraneAt(Left) (L) cũng có mặt. Cần kiểm tra On(Cargo,Truck) và CraneAt(Left) có mutex nhau ở S2 không trước khi trích plan — xem câu d). **d) Mutex ở hai mức đầu tiên (A0)**

Cặp hành động	Loại mutex	Giải thích
Move(Left,Right) & PickUp(Left)	Interference	Move xóa CraneAt(Left) (delete list), trong khi CraneAt(Left) là precondition của PickUp(Left).
Move(Left,Right) & Noop(CraneAt(Left))	Inconsistent Effects	Move xóa CraneAt(Left) trong khi Noop khẳng định CraneAt(Left) tiếp tục đúng — hai effect mâu thuẫn trực tiếp.
PickUp(Left) & Noop(CargoAt(Left))	Inconsistent Effects	PickUp xóa CargoAt(Left), Noop khẳng định CargoAt(Left) vẫn đúng.
PickUp(Left) & Noop(Empty(Crane))	Inconsistent Effects	PickUp xóa Empty(Crane), Noop khẳng định Empty(Crane) vẫn đúng.

Hệ quả ở S1: vì Move và PickUp mutex nhau (chỉ có một nguồn sinh ra mỗi bên), CraneAt(Right) và Holding(Crane,Cargo) cũng mutex nhau tại S1 (Competing Needs kế thừa). Đến A1, hành động LoadOnTruck(Right) cần cả CraneAt(Right) lẫn Holding(Crane,Cargo) làm precondition — hai precondition này mutex nhau ở S1, nên LoadOnTruck(Right) tại A1 mang theo Competing Needs với chính cặp tiền điều kiện của nó; tuy nhiên đến mức sau (A2/S3 nếu mở rộng thêm), khi Move và PickUp đã có thể thực hiện lần lượt ở các mức khác nhau (ví dụ PickUp ở A0, Move ở A1), CraneAt(Right) và Holding đều có thể cùng đạt được từ một nguồn không-mutex, nên mutex này sẽ biến mất và LoadOnTruck trở nên khả thi — minh họa đúng cơ chế "mutex có thể mất dần qua các mức" của GraphPlan.

Bài 3. Quan hệ Mutex trong planning graph (bài toán Pinky)

Đối chiếu đúng planning graph đã cho trong đề (S0={At(Home),Rich} → A0={Move(Home,CC)} → S1={At(Home),At(CC),Rich} → A1={Move(Home,CC),Move(CC,Home),Buy(Cheese),Gamble} → S2={At(Home),At(CC),Has(Cheese),Rich,¬Rich} → A2={Move(Home,CC),Move(CC,Home),Buy(Cheese),Eat(Cheese),Gamble,ATM} → S3 chứa Full).

a) Mutex do Inconsistent Effects ở mức A0

Cặp: Move(Home,CC) & Noop(At(Home)). Move(Home,CC) xóa At(Home) (delete list) trong khi Noop(At(Home)) khẳng định At(Home) tiếp tục đúng ở S1 — hai effect trái ngược trực tiếp trên cùng literal At(Home).

b) Mutex do Interference ở mức A1

Cặp: Move(CC,Home) & Buy(Cheese). Move(CC,Home) xóa At(CC) (delete list), trong khi At(CC) là precondition bắt buộc của Buy(Cheese) (cần At(ChuckyCheese)). Thực hiện đồng thời sẽ làm Buy(Cheese) mất tiền điều kiện ngay khi áp dụng.

c) Mutex do Competing Needs ở mức A2

Cặp: Buy(Cheese) & ATM. Buy(Cheese) cần precondition Rich; ATM cần precondition ¬Rich. Rich và ¬Rich là một cặp literal phủ định nhau nên luôn mutex tại S2 (quy tắc nền tảng: một literal và phủ định của nó luôn mutex) — vì vậy hai hành động cần hai tiền điều kiện loại trừ nhau này cũng mutex theo Competing Needs, dù bản thân Buy(Cheese) và ATM không trực tiếp xóa effect của nhau.

d) Giải thích vì sao mỗi cặp thỏa đúng loại mutex

- Move(Home,CC) & Noop(At(Home)): thỏa Inconsistent Effects vì hai effect (một xóa, một giữ) mâu thuẫn trực tiếp trên cùng một literal duy nhất, không liên quan đến precondition của hành động nào.
- Move(CC,Home) & Buy(Cheese): thỏa Interference (không phải Inconsistent Effects) vì Buy(Cheese) không hề có effect nào về At(CC) — nó chỉ bị ảnh hưởng vì mất đi PRECONDITION, đúng định nghĩa Interference.
- Buy(Cheese) & ATM: thỏa Competing Needs vì hai hành động này không hề chạm vào effect của nhau (Buy không xóa gì liên quan ATM và ngược lại) — nguyên nhân mutex nằm hoàn toàn ở chỗ PRECONDITION của chúng (Rich và ¬Rich) xung khắc nhau từ mức trước, đúng định nghĩa Competing Needs.

Bài 4. Classical Planning và GraphPlan — Valet Parking

Trạng thái ban đầu (S0): ParkedBehind(B,A), ParkedBehind(A,Curb), ¬ParkedBehind(B,Curb), ParkedBehind(C,Curb), ClearBehind(B), ClearBehind(C). Suy ra bố cục: làn 1 (từ lề ra ngoài): Curb– A– B; làn 2: Curb–C (còn trống phía sau C).

a) Hành động hợp lệ ở mức A0 (ô 1–4)

Xét mọi cách gán biến hợp lệ cho ParkBehind(car1,car2) và ParkInNewRow(car) với car {A,B,C}:∈

Ô	Hành động	Kiểm tra precondition	Kết luận
1	ParkBehind(B,C)	ClearBehind(B)✓, ClearBehind(C)✓, ParkedBehind(B,A)✓ (place=A)	Hợp lệ
2	ParkBehind(C,B)	ClearBehind(C)✓, ClearBehind(B)✓, ParkedBehind(C,Curb)✓ (place=Curb)	Hợp lệ
3	ParkInNewRow(B)	ClearBehind(B)✓, ParkedBehind(B,A)✓, ¬ParkedBehind(B,Curb)✓	Hợp lệ

4	ParkInNewRow(C)	ClearBehind(C)✓, ParkedBehind(C,Curb)✓, nhưng ¬ParkedBehind(C,Curb) SAI (C đang ở Curb)	N/A

(Các phương án còn lại đều loại vì ClearBehind(A) sai — A đang bị B chắn phía sau — nên A không thể là car1/car2/car trong bất kỳ toán tử nào ở trên.)

b) Predicate ở S1 nhận add-edge từ mỗi hành động

Ô	Add-edge tới S1
1	ParkedBehind(B,C); ClearBehind(A); ¬ParkedBehind(B,A); ¬ClearBehind(C)
2	ParkedBehind(C,B); ClearBehind(Curb); ¬ParkedBehind(C,Curb); ¬ClearBehind(B)
3	ParkedBehind(B,Curb); ClearBehind(A); ¬ParkedBehind(B,A)
4	N/A

c) Mutex do Inconsistent Effects

ParkBehind(B,C) [ô 1] & Noop(ParkedBehind(B,A)): ô 1 thêm ¬ParkedBehind(B,A) (tức xóa ParkedBehind(B,A)), trong khi Noop khẳng định ParkedBehind(B,A) vẫn đúng ở S1 — mâu thuẫn trực tiếp trên cùng literal.

d) Mutex do Interference

ParkBehind(C,B) [ô 2] & ParkBehind(B,C) [ô 1]: ô 2 xóa ClearBehind(B) (delete list), trong khi ClearBehind(B) là precondition bắt buộc của ô 1 (và cũng của ô 3) — ô 2 làm mất tiền điều kiện mà ô 1 cần.

e) GraphPlan "leveled off" khi nào?

Đáp án đúng: C. Hai mức liên tiếp của planning graph giống hệt nhau (cả về tập literal, tập hành động và tập các cặp mutex không còn thay đổi giữa hai mức liên tiếp).

Bài 5. Xây dựng GraphPlan và trích xuất kế hoạch

S0 = {A}. Không toán tử nào có delete list, ngoại trừ op6 (add E, ¬A — tức xóa A).

a) Các mức của planning graph

Mức	Nội dung
S0	A
A0	op1(→B), op3(→C), op4(→D), op5(→E), op6(→E,¬A); Noop(A). (op2 chưa áp dụng được vì cần B)
S1	A, ¬A, B, C, D, E — đã đủ cả 3 mục tiêu C, D, E

Kiểm tra mutex giữa C, D, E tại S1: op3 (sinh C), op4 (sinh D), op5 (sinh E) đều chỉ có precondition A, không toán tử nào có delete list → không có Interference lẫn Inconsistent Effects giữa 3 toán tử này, và tiền điều kiện của chúng (đều là A) không mutex với chính nó → C, D, E ĐÔI MỘT KHÔNG MUTEX tại S1. Mục tiêu C D E đã đạt được, không mutex, ngay tại mức S1 đầu tiên.∧ ∧

b) Kế hoạch được GraphPlan trả về

{op3, op4, op5} — ba hành động này không mutex nhau nên có thể xếp thực hiện ĐỒNG THỜI trong cùng một bước thời gian (time step) duy nhất tại A0, đạt C D E ngay ở S1. (Nếu diễn giải theo đúng∧ ∧ câu dẫn của đề — mỗi thời điểm chỉ 1 toán tử — thì đây là 3 bước tuần tự thực hiện op3, op4, op5 theo thứ tự bất kỳ, vì ba hành động độc lập hoàn toàn với nhau.)

c) Kế hoạch trên có tối ưu về số lượng thao tác không?

KHÔNG.

Kế hoạch {op3,op4,op5} tối ưu về số MỨC/thời gian (chỉ cần 1 time step nếu cho song song), nhưng xét về TỔNG SỐ THAO TÁC thì chưa tối ưu: dãy {op1 → op2} chỉ cần 2 hành động (op1 cho B, rồi op2 dùng B tạo ra C, D, E cùng lúc) để đạt đúng mục tiêu C D E, ít hơn 3 hành động của phương ánΛ Λ op3+op4+op5. Đây chính là đặc điểm quan trọng cần ghi nhớ của GraphPlan: thuật toán đảm bảo tìm ra kế hoạch với SỐ MỨC THỜI GIAN (số bước song song) ít nhất, chứ KHÔNG đảm bảo tổng số hành động là nhỏ nhất — vì GraphPlan dừng lại ngay khi mục tiêu xuất hiện không-mutex ở mức sớm nhất (S1), nên không có cơ hội khám phá tới mức S2 để tìm ra phương án {op1,op2} ít thao tác hơn nhưng tốn 2 time step.

d) Mutex ở A0 và mutex proposition ở S1

Cặp toán tử mutex ở A0	Loại mutex	Lý do
op6 & op1	Interference	op6 xóa A, là precondition của op1
op6 & op3	Interference	op6 xóa A, là precondition của op3
op6 & op4	Interference	op6 xóa A, là precondition của op4
op6 & op5	Interference	op6 xóa A, là precondition của op5
op6 & Noop(A)	Inconsistent Effects	op6 thêm ¬A trong khi Noop(A) thêm A — mâu thuẫn trực tiếp

(op1, op3, op4, op5 đôi một KHÔNG mutex với nhau — không có delete list nên không thể Interference/Inconsistent Effects, và cùng chia sẻ một precondition A duy nhất nên không Competing Needs.)

Cặp proposition mutex ở S1	Lý do
A & ¬A	Luôn mutex vì là một cặp literal phủ định nhau (quy tắc nền tảng)
B & ¬A	Nguồn sinh B chỉ có op1; nguồn sinh ¬A chỉ có op6; cặp (op1,op6) mutex → B,¬A mutex
C & ¬A	Nguồn sinh C chỉ có op3; (op3,op6) mutex → C,¬A mutex
D & ¬A	Nguồn sinh D chỉ có op4; (op4,op6) mutex → D,¬A mutex

Lưu ý: E & ¬A KHÔNG mutex, dù (op5,op6) không được xét vì op6 TỰ NÓ sinh ra cả E lẫn ¬A cùng lúc — khi một hành động duy nhất tạo ra cả hai literal, chúng mặc nhiên không mutex (có ít nhất một cách đạt đồng thời).

Bài 6. Tổng hợp và phân tích

a) So sánh tìm kiếm trực tiếp với GraphPlan

Tìm kiếm trực tiếp trong không gian trạng thái duyệt từng trạng thái cụ thể, mỗi bước áp dụng đúng một hành động, nên với nhiều hành động độc lập có thể xảy ra song song, nó phải thử mọi thứ tự/hoán vị của chúng, dẫn tới bùng nổ tổ hợp. GraphPlan xây một cấu trúc nén biểu diễn đồng thời mọi literal/hành động khả dĩ ở từng mức, dùng mutex để loại bỏ sớm tổ hợp không khả thi và cho phép gộp nhiều hành động không mutex vào CÙNG một mức. Planning graph giúp giảm đáng kể không gian tìm kiếm nhất là khi bài toán có NHIỀU hành động độc lập, không tương tác (không mutex) có thể thực hiện song song — khi đó số mức cần thiết để đạt mục tiêu nhỏ hơn nhiều so với độ sâu của cây tìm kiếm tuần tự. **b) Vai trò của No-op**

No-op cho phép một literal đã đúng ở mức S_i tiếp tục được công nhận là đúng ở mức S(i+1) mà không cần hành động nào thực sự tác động — đây là cách GraphPlan biểu diễn "frame assumption" (một fact không tự nhiên biến mất nếu không ai xóa nó). Nếu bỏ toàn bộ No-op, một literal chỉ tồn tại đúng ở mức mà một hành động vừa tạo ra nó rồi biến mất ngay ở mức kế tiếp; các precondition cần dùng lại một fact có từ trước đó nhiều mức sẽ không còn đường add-edge nào để "theo kịp" tới đúng mức cần dùng, khiến đồ thị không phản ánh đúng những fact cần được duy trì, và việc trích xuất kế hoạch sẽ thất bại hoặc sai ngay cả khi lời giải thực sự tồn tại.

c) Vai trò của mutex giữa các literal mục tiêu

Nếu tại một mức, các literal mục tiêu đôi một không còn mutex, GraphPlan tiến hành trích xuất kế hoạch ngay tại mức đó. Nếu vẫn còn ít nhất một cặp mục tiêu mutex, thuật toán KHÔNG kết luận thất bại ngay mà mở rộng thêm một mức nữa (vì mutex hoàn toàn có thể biến mất ở mức sau, khi xuất hiện thêm cách đạt literal không mutex). Chỉ khi đồ thị đã "leveled off" (ổn định, hai mức liên tiếp giống hệt nhau) mà cặp mục tiêu đó VẪN còn mutex, GraphPlan mới kết luận chắc chắn KHÔNG tồn tại kế hoạch (bài toán vô nghiệm) — vì từ đó về sau mutex này sẽ không bao giờ mất đi nữa.

d) Quy trình trích xuất kế hoạch từ planning graph

- 1. Tại mức S hiện tại với tập mục tiêu G: với mỗi literal trong G, chọn một hành động (hoặc $Noop_k$) ở A_{k-1} có add-edge tới literal đó, sao cho toàn bộ các hành động được chọn cho mọi literal trong G đôi một không mutex (nếu không chọn được tổ hợp hợp lệ, quay lui thử lựa chọn khác).
- 2. Gộp tất cả precondition của các hành động vừa chọn (bỏ qua Noop) thành tập mục tiêu MỚI cần đạt ở mức S_{k-1}.
- 3. Lặp lại đệ quy lùi dần về các mức trước (k−1, k−2, …) với tập mục tiêu mới ở mỗi bước, cho đến khi tới S0.
- 4. Nếu tập mục tiêu tại S0 là tập con của trạng thái ban đầu thì quá trình thành công; kế hoạch cuối cùng là dãy các tập hành động (bỏ Noop) đã chọn tại mỗi mức, theo thứ tự từ A0 đến A_{k-1} — các hành động cùng một mức có thể thực hiện song song, các mức khác nhau thực hiện tuần tự.